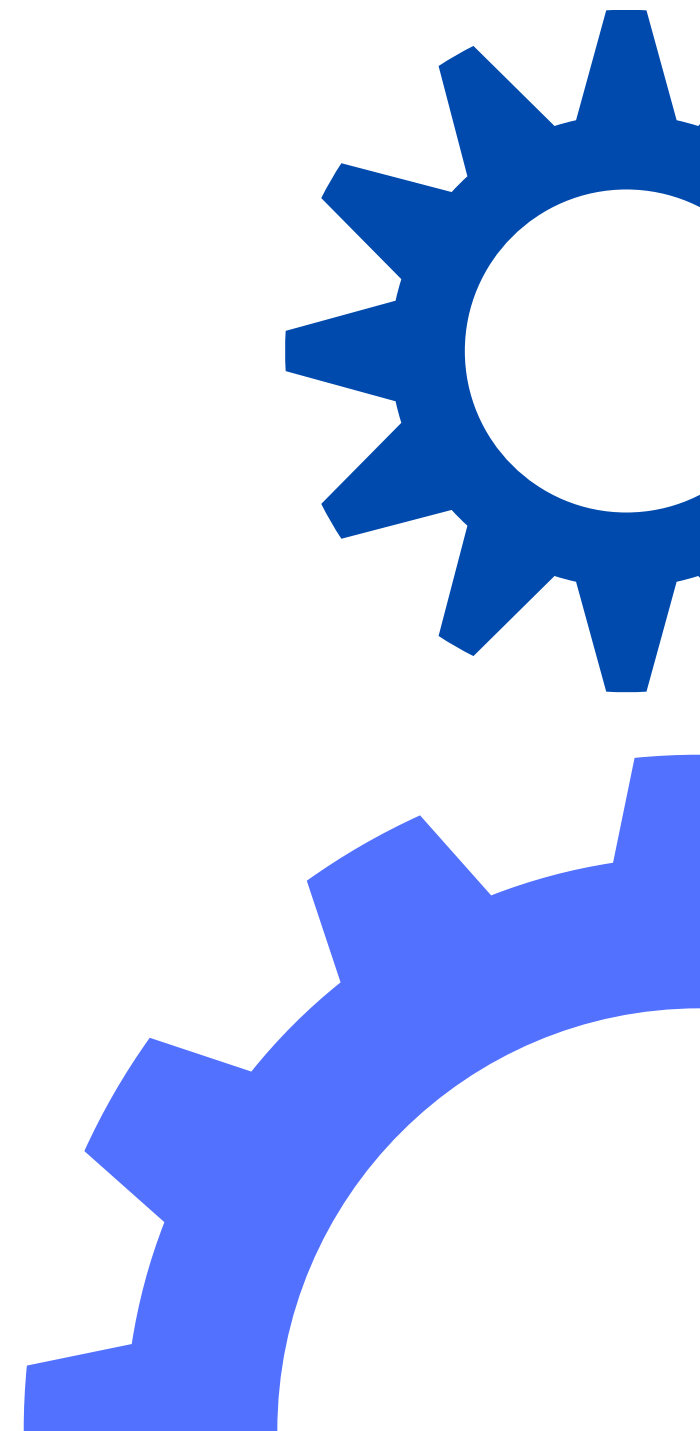


MLOps Case Study:

Team members

Shalini P	-	1RV23AI088
Shanthesh A S	-	1RV23AI090
Shashank K	-	1RV23AI091
Shravyaa S	-	1RV23AI093



QUESTION 5

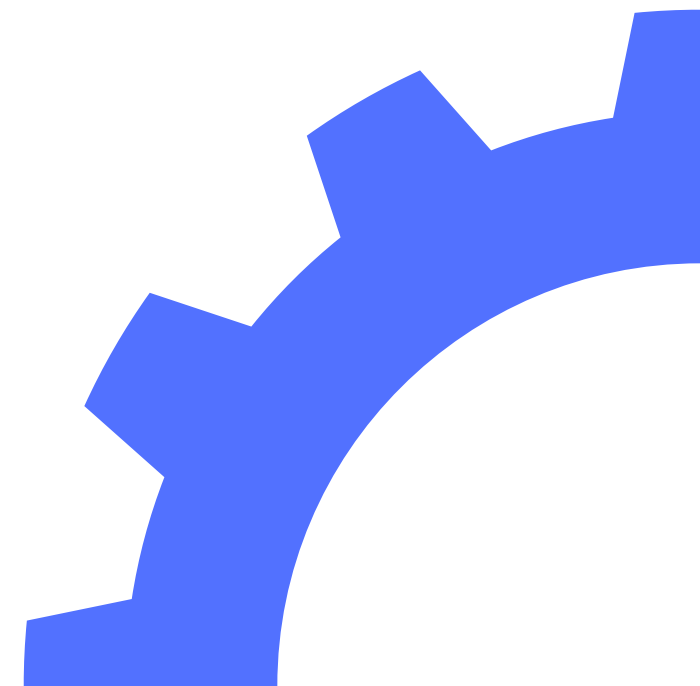
How can MedInsure ensure full reproducibility of the model? Describe the role of versioning training data, storing environment metadata, and tracking hyperparameters.



Model reproducibility

Storing details about the model like the version of data used, the version of environment the model was trained on, data preprocessing steps, training environment (hardware, libraries, OS), random seeds, hyperparameters, model architecture, and training code, etc. that can be used later to rebuild the model from scratch such that it has the exact same results is called Model Reproducibility.

It ensures experiments can be reliably repeated, verified, and compared by others.



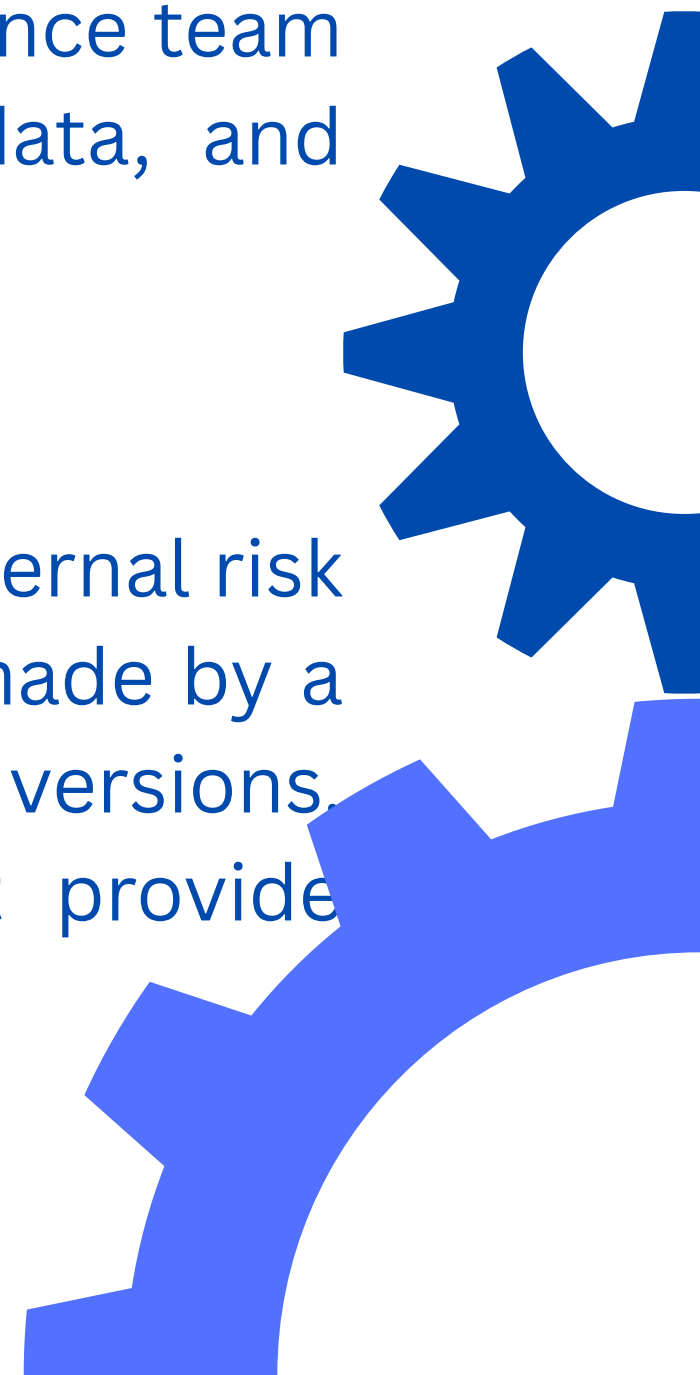
Model reproducibility: WHY

- Version rollback during model revisions:

During testing and validation, MedInsure's claim-approval model is frequently fine-tuned for example, to improve performance on specific claim types like rare surgical procedures. Some of these revisions may reduce accuracy or introduce bias. Reproducibility ensures that if a newer version performs worse, the data science team can revert to the last best model with the exact same environment, data, and hyperparameters.

- Regulatory and audit compliance:

Health insurance models operate in a highly regulated domain. Auditors or internal risk evaluators may request to replay previous training runs or verify decisions made by a specific model version. Without reproducible records — including dataset versions, preprocessing pipelines, and environment metadata — the team cannot provide evidence of fair and consistent decision-making for past claim approvals.



How can MedInsure ensure full reproducibility of the model?

MedInsure should store every version of the training and validation datasets, including the date of extraction, schema, and preprocessing steps.

The environment and language versions and ML library versions should be recorded in detail. The team should containerize the training and deployment environments using Docker, ensuring the same dependencies are replicated anywhere.

Every training run should automatically log the model architecture, hyperparameters, random seeds, and training metrics using tools like MLflow Tracking.

Store preprocessing scripts, feature-engineering code, and model training notebooks in a Git repository with version tags that align with model versions.



Tools used

Issue 1: Predictions differed between development and staging environments (Python 3.8 vs 3.10, library mismatch).

→ Tool Used: Docker / Conda Environments

MedInsure can containerize its model training and deployment setup so that the same Python version, ML libraries (e.g., scikit-learn, pandas), and OS dependencies are preserved across all environments. This eliminates inconsistencies in claim-score predictions.

Issue 2: Auditor requested proof of training parameters, but the team could not reproduce the exact run.

→ Tool Used: MLflow Tracking & Weights & Biases (W&B)

By logging all hyperparameters, seed values, and model metrics in MLflow, MedInsure can reproduce and verify the exact configuration used for each model version during audits.

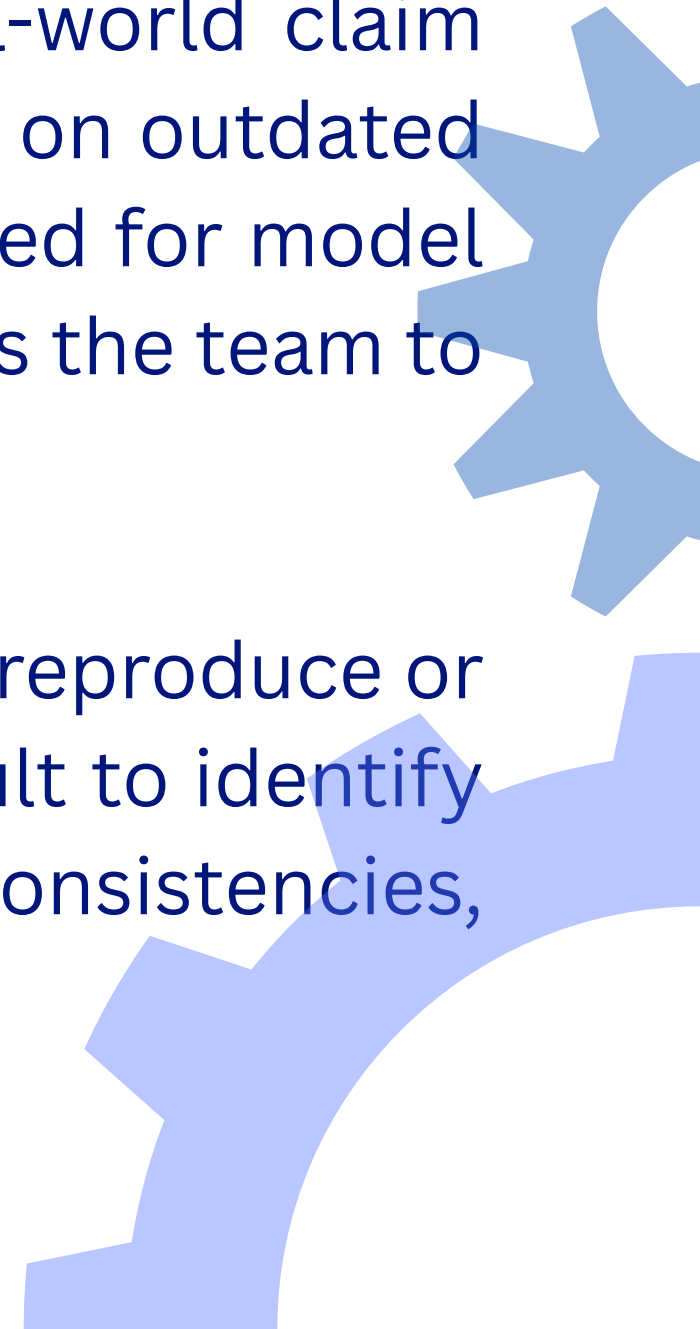


Role of versioning the training data

The data used for MedInsure's claim-approval model includes patient demographics, hospital information, diagnosis and treatment details, claim amounts, and fraud-risk indicators.

Regularly updating the training data helps prevent input drift — where real-world claim patterns change, causing the model to make inaccurate predictions if it relies on outdated data. Versioning the training data means saving and labelling each dataset used for model training with its exact timestamp, schema, and preprocessing steps. This allows the team to trace which version produced a particular model result.

If training data versions are not recorded, MedInsure would lose the ability to reproduce or explain past model decisions, especially during audits. It would also be difficult to identify which data updates improved or degraded performance, leading to inconsistencies, compliance risks, and unreliable model retraining.



Role of storing environment meta-data

To ensure full reproducibility, MedInsure must capture and store the complete environment metadata used during model training. This includes Python version, library versions, OS details, hardware configurations, and all dependency files. Without this, any small change in the environment can alter model behavior.

In the case study, predictions differed between the development notebook and the staging server because Python 3.8 and 3.10 handled certain ML libraries differently. If environment metadata was stored, the team could have recreated the exact setup and avoided these inconsistencies.

By maintaining versioned environment snapshots using Docker, Conda environment files, or MLflow environment tracking, MedInsure ensures that the model can be recreated, audited, and validated at any time with identical results.

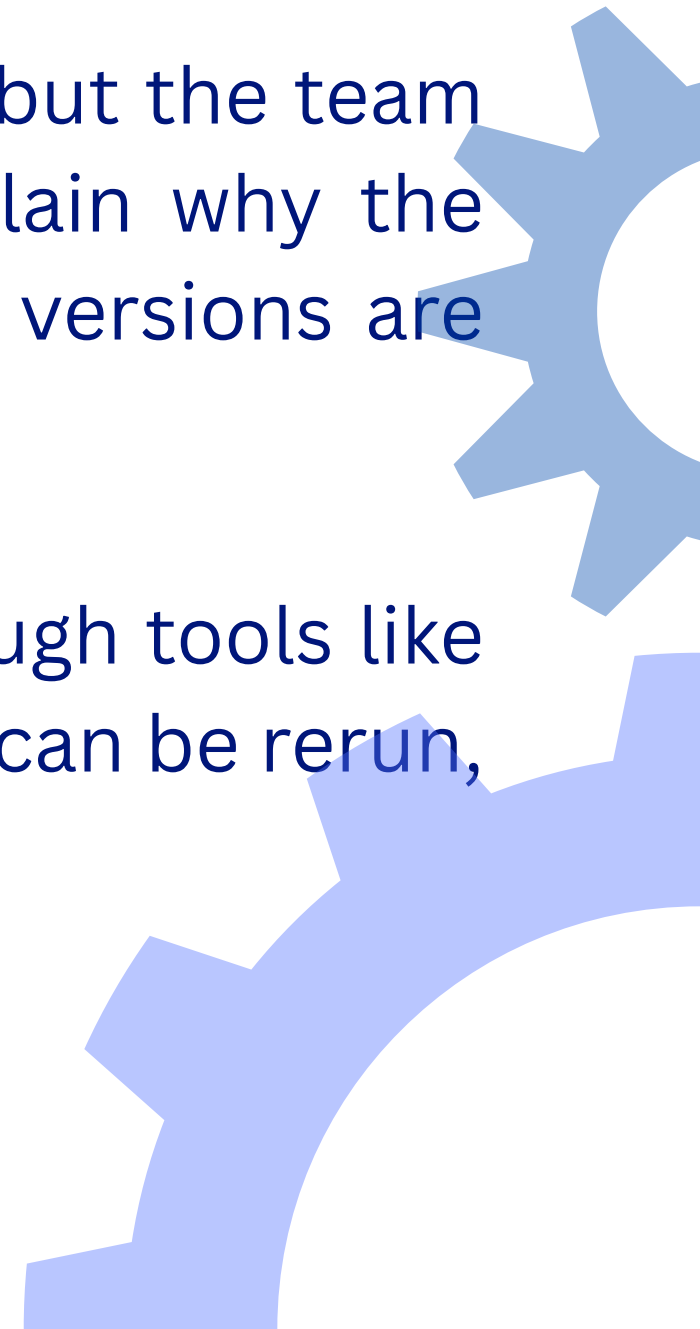


Role of tracking hyperparameters

Hyperparameters define how the model learns learning rate, regularization, class weights, sampling strategy, and more. Tracking these during training is essential because even minor changes can lead to significantly different predictions and risk scores.

In MedInsure's case, auditors requested proof of the hyperparameters used, but the team couldn't reproduce the exact training run. This makes it impossible to explain why the model behaves poorly on rare surgical procedures or validate whether new versions are genuinely improved.

By logging all hyperparameters, seed values, and training configurations through tools like MLflow or Weights & Biases, MedInsure can guarantee that any model version can be rerun, compared, and audited with full transparency and reliability.



QUESTION 3

Identify the sources of model risk in this scenario. How does poor performance on rare surgical claims illustrate the importance of evaluating model robustness across subpopulations?

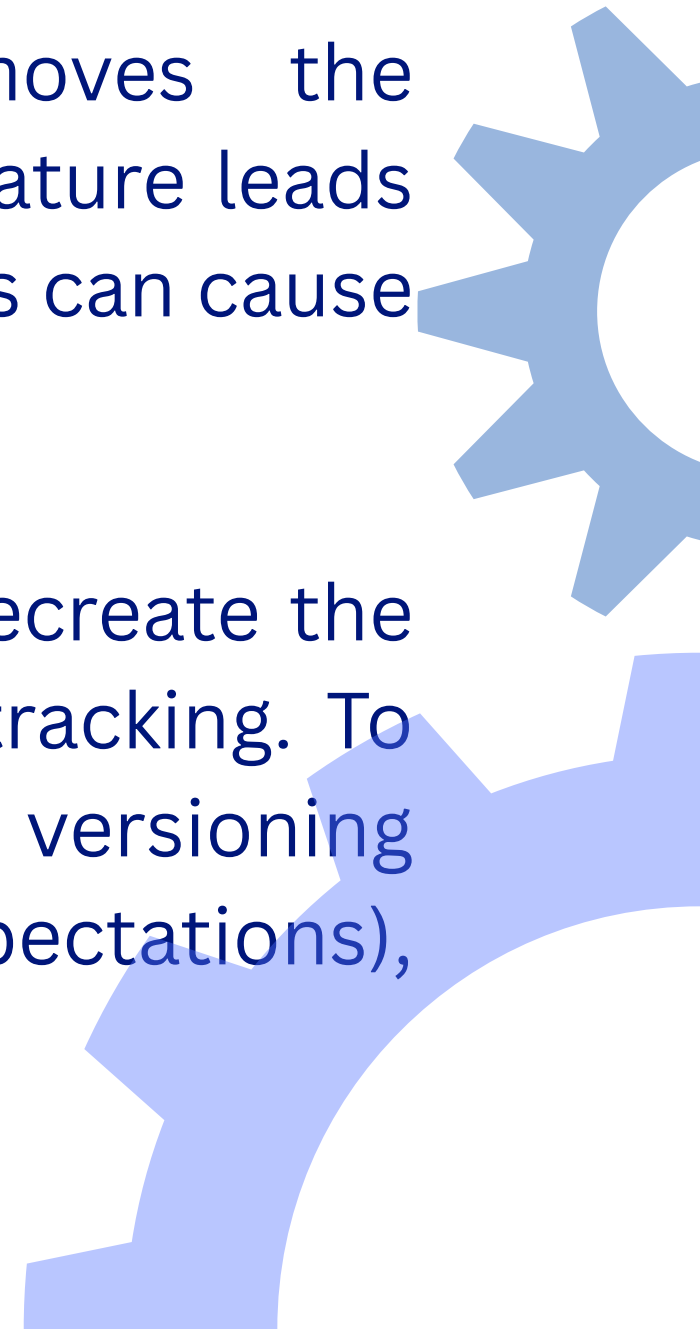


Source of model risk

The model was trained in Python 3.8 but deployed in Python 3.10 with different ML library versions, causing inconsistent predictions. This creates prediction drift and makes the model unreliable across environments.

Data-related risk appears when the updated ETL pipeline removes the “hospital_risk_score” feature, which the model still expects. The missing feature leads to 22% claim rejections, showing how schema drift and broken data pipelines can cause operational failures.

Governance and reproducibility risk is present because the team cannot recreate the exact training run due to missing environment logs and hyperparameter tracking. To solve these issues technically, MedInsure should implement strict data versioning (DVC), environment packaging (Conda/Docker), schema validation (Great Expectations), and adversarial robustness checks to detect manipulated inputs.



Importance of model robustness

The model performs poorly on claims involving rare surgical procedures because these cases were underrepresented in the training dataset. As a result, the model never learned strong decision boundaries and therefore produces weak or biased predictions.

Even though overall accuracy looked strong during validation, the model failed to generalize to smaller but high-impact claim categories. For MedInsure, this creates financial risk (incorrect approvals), regulatory risk (unfair or inconsistent decisions), and business-risk exposure. It highlights why high global accuracy does not guarantee fairness or reliability across all claim groups.

MedInsure can fix this by performing stratified evaluation across subgroups, oversampling or SMOTE for rare procedures, reweighting loss functions, and applying model monitoring to track subpopulation performance in production. These steps ensure the model becomes robust not just on majority claim types but across all medically important subgroups.

